

Project 7 (Bonus) — MCPForge-Local-Notes-And-Tasks-Server

Title: MCPForge Local Notes And Tasks Server

Type: MCP Server (2 Tools, 1 Resource, 1 Prompt) · **Stack:** Python, official `mcp` SDK (MCPServer), SQLite, MCP Inspector · **Level:** Intermediate (bonus project) **Tagline:** A real MCP server — Local Notes & Tasks — exposing 2 Tools, 1 Resource, and 1 Prompt over the Model Context Protocol, usable by any MCP-compatible client without a custom integration for each one.

Overview

MCPForge is a real MCP (Model Context Protocol) server that exposes a small "Local Notes & Tasks" capability — adding notes, searching them, reading them, and summarizing them — over one standard protocol instead of hardcoded into a single app. Any MCP-compatible client (Claude Desktop, an IDE plugin, your own agent code) can connect to it, discover what it offers at runtime, and use it without a custom integration. This solves the real problem of the same tool logic getting copy-pasted and re-wired into every app that wants it, drifting out of sync the moment one copy changes — anyone who needs one capability shared cleanly across multiple AI clients would want something built this way.

Features

- Exposes `add_note` and `search_notes` as MCP Tools, backed by a real SQLite database that survives a server restart
- Exposes `notes://all` as a read-only MCP Resource, kept separate from Tools since reading it changes nothing
- Exposes `summarize_notes` as a reusable, parameterized MCP Prompt template instead of a string hardcoded into one app
- Validates input and rejects empty or whitespace-only notes with a clear error instead of silently storing them
- Ships with a standalone MCP client script that connects, discovers tools/resources/prompts at runtime, and exercises all three building blocks end to end
- Testable interactively via the official MCP Inspector before any custom client code is written
- Runs over stdio, usable by Claude Desktop, an IDE, or any other MCP-compatible client with zero custom integration code

Tech Stack

- Python
- Official `mcp` SDK (MCPServer)
- SQLite
- MCP Inspector

Prerequisites

- Comfortable with basic config and logging setup in a Python project
- Understands a typed argument shape for a function, and clean error handling instead of a crash
- Familiar with the idea of tool-calling, and MCP's three building blocks: Tools, Resources, and Prompts
- Comfortable running a local SQLite database from Python

Architecture

A `MCPServer` server process communicates over stdio with any connecting MCP client. It exposes four things behind one protocol: two Tools (`add_note` , `search_notes`) that read and write a local SQLite database, one Resource (`notes://all`) that hands back a read-only listing with no side effects, and one Prompt (`summarize_notes`) that returns a filled-in template for a client to send to its own model. A client — the MCP Inspector for manual testing, or the project's own `client.py` — connects, calls `list_tools()` / `list_resources()` / `list_prompts()` to discover what's on offer, and then calls whichever piece it needs by name, with no hardcoded knowledge of the server's internals.

Setup (do this once, before Step 1)

These are the very first commands to run. You don't need to read anything else first.

```
cd project_7_mcpforge
python -m venv .env
source .env/bin/activate          # Windows: .env\Scripts\activate
pip install "mcp[cli]" python-dotenv pydantic
pip freeze > requirements.txt
```

Create a file called `.env` in this folder (never commit this file to git):

```
LOG_LEVEL=INFO
NOTES_DB_PATH=notes.db
```

Create a second file, `.env.example`, with the same keys:

```
LOG_LEVEL=INFO
NOTES_DB_PATH=notes.db
```

Neither key here is a secret, so real-looking values are fine in both files — this project never touches an LLM API key at all, since the server itself doesn't call a model.

Add `.venv/` and `notes.db` to your `.gitignore` before your first commit — the database file is generated locally by Step 2, not something to commit.

`mcp[cli]` installs the **MCP Inspector**, a small local web UI that connects to your server, lists its Tools/Resources/Prompts, and lets you call them by hand — the fastest way to test a server before writing any client code of your own. Run it against your server with `mcp dev server.py`.

Troubleshooting

Problem	Fix
The MCP Inspector connects, but <code>add_note</code> doesn't show up in the tool list	Check the function is actually decorated with <code>@mcp.tool()</code> , and that <code>mcp.run(transport="stdio")</code> is the last line that actually runs — a server that never calls <code>mcp.run()</code> never starts listening
<code>add_note("")</code> silently creates a blank note	Validate <code>text.strip()</code> before touching the database, and raise a clear error if it's empty — the standard "fail loudly, not silently" idea, applied here to a tool's own input content, not just to missing config
Notes disappear every time you restart the server	You're still using Step 1's in-memory list. Step 2 replaces it with a real SQLite file (<code>sqlite3.connect(db_path)</code>), so notes survive a restart
<code>search_notes("Dentist")</code> finds nothing, even though a note says "dentist"	Your search is doing an exact, case-sensitive match. Use SQL <code>LIKE</code> with <code>%query%</code> , and lowercase both sides before comparing
The client hangs forever and never gets a reply	Something inside the server is writing to stdout directly (a stray <code>print()</code>) — on the stdio transport, stdout is the protocol channel. Route all logging to stderr (or a file) through <code>logging_setup.py</code> , never <code>print()</code>

Problem	Fix
You can't decide whether something should be a Tool or a Resource	Ask one question: does calling it <i>change</i> anything, or cost real computation? If yes, it's a Tool. If it's only handing back data to read, it's a Resource

Usage Example

The Tool/Resource/Prompt part of this project needs a real example to work with — not just "store some notes." Use this one, or make your own, but keep it this specific:

Scenario: a small local notes tool. Each note is just free text plus when it was created.

Test these 3 inputs for `add_note`: 1. "Buy milk and eggs" → succeeds, returns something like "Note added with id 1." 2. "Call the dentist to reschedule" → succeeds, returns "Note added with id 2." 3. "" (empty string) or " " (only whitespace) → must be **rejected** with a clear error, not silently stored as a blank note.

Then: - `search_notes("dentist")` → returns ["Call the dentist to reschedule"] — note 1 doesn't match, and note 3 was never stored at all. - Reading the `notes://all` Resource → returns a read-only listing of both real notes, with their ids. - Using the `summarize_notes` Prompt with a date range covering today → returns a filled-in prompt *template* asking a model to summarize the notes created in that range. The server produces the prompt text, not a summary — whichever client you're using is the one that actually sends that prompt to a model.

Why input 3 matters: it's the same "don't let bad input slide through" lesson a missing required field teaches elsewhere — here it's an empty *required* field instead, and the fix takes the same instinct: catch it on purpose, fail with a clear message, and never store something you already know is invalid.

Jump to: [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#)

How To Build This — Step by Step

Step 1 — A Minimal Server With One Tool

Project: MCPForge-Local-Notes-And-Tasks-Server — Step 1 of 4: A Minimal Server With One Tool

What this step does: proves the most basic thing works — your code can start an MCP server, and a client (the Inspector, or a tiny script) can connect to it and successfully call its one tool. Nothing else yet. **Why this step matters:** every layer you add after this (real persistence, Resources, Prompts) is worthless if the basic protocol handshake doesn't work — this step isolates that handshake from everything else, so if something breaks later, you already know the connection itself isn't the cause. **When you'll hit this for real:** the first thing you should do on *any* new MCP server,

professional or personal — before adding real logic, prove a client can actually connect and call the simplest possible tool. **Helpful background:** config and logging basics, what MCP actually is and the problem it solves, and how MCP servers and clients actually connect.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Build `config.py` and `logging_setup.py` (or reuse them if you already built similar modules elsewhere). `LOG_LEVEL` is the only setting you need this step. 2. Install the official `mcp` SDK (`pip install "mcp[cli]"`) and write `server.py` : a `MCPServer` instance, with one tool, `add_note(text: str) -> str`, that appends to a plain in-memory Python list (no database yet) and returns a confirmation string. 3. Run the server with the MCP Inspector (`mcp dev server.py`) and call `add_note` by hand through its UI — confirm it responds correctly. If you'd rather not use the Inspector, write a short client script using `stdio_client / ClientSession` (the standard MCP client connection shape) that calls `add_note` once and prints the result. 4. Add a note twice in the same session and confirm both calls succeed — this in-memory list is expected to forget everything the moment the server process stops. That's the exact problem Step 2 fixes.

Your files after Step 1:

```
project_7_mcpforge_mcp_server/
├─ .env / .env.example      → LOG_LEVEL (a reused config pattern)
├─ config.py                (a reused config module)
├─ logging_setup.py         (a reused logging module)
└─ server.py                → MCPServer("notes-server"); one tool: add_note(te
```

Step 2 — A Real SQLite-Backed Store, and the Search Tool

Project: MCPForge-Local-Notes-And-Tasks-Server — Step 2 of 4: A Real SQLite-Backed Store, and the Search Tool

What this step does: replaces the in-memory list with a real SQLite database, so notes survive a server restart, and adds the second Tool, `search_notes`. **What's new vs. Step 1:** a `db.py` module with real persistence, an `exceptions.py` with a custom error for invalid input, and input validation on `add_note` that rejects empty/whitespace-only text instead of silently storing it. **What stays the same:** the server still only exposes Tools — Step 3 is where the Resource and the Prompt show up. **Why this step matters:** an MCP server that forgets everything on restart isn't something a real client would actually rely on — and a tool that silently accepts garbage input (an empty note) is exactly the kind of bug the standard "fail loudly, not silently" idea, and the usual Pydantic-argument-checking idea, both exist to prevent, just applied one level deeper than the argument's *shape* — this time it's the argument's *content* that needs checking. **When you'll hit this for real:** any tool backed by real storage — a note, a task, a database row — needs this exact pair of fixes: real persistence, and validation that rejects bad input before it's ever written. **Helpful background:** checking arguments with Pydantic as the contract, and getting a tool's failure back to the model correctly.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Add `NOTES_DB_PATH` to your config. Write `db.py : init_db(path)` (creates the `notes` table if it doesn't exist), `insert_note(path, text)`, and `find_notes(path, query)` using SQL `LIKE` for a real substring search. 2. Write `exceptions.py` → `EmptyNoteError(Exception)`. In `server.py`'s `add_note`, check `text.strip()` before calling `insert_note()` — raise `EmptyNoteError` with a clear message if it's empty, and let the tool's `except` block turn that into a clean error string the client sees (the same pattern as the standard tool-failure-handling approach), not a crash. 3. Add the second tool, `search_notes(query: str) -> list[str]`, calling `find_notes()`. 4. Test: add 2-3 real notes, restart the server process entirely, and confirm `search_notes` still finds them — this is the actual proof persistence works, not just that the code runs once. Then call `add_note("")` and `add_note(" ")` and confirm both are rejected with a clear message, not silently stored.

Your files after Step 2:

```
project_7_mcpforge_mcp_server/
├─ .env / .env.example
├─ config.py                → now also loads NOTES_DB_PATH
├─ logging_setup.py
├─ exceptions.py            → EmptyNoteError(Exception)
├─ db.py                   → init_db(), insert_note(), find_notes()
└─ server.py               → add_note() and search_notes() tools, both ba
```

Step 3 — Add the Resource and the Prompt

Project: MCPForge-Local-Notes-And-Tasks-Server — Step 3 of 4: Add the Resource and the Prompt

What this step does: adds the Resource (`notes://all`) and the Prompt (`summarize_notes`) — the point isn't the code, which is small, it's deciding correctly which of the 4 things you're building belongs on which of the 3 MCP building blocks. **What's new vs. Step 2:**

`@mcp.resource("notes://all")` and `@mcp.prompt()` decorated functions in `server.py`.

What stays the same: `add_note` and `search_notes` don't change at all — they're staying Tools, on purpose, because they clearly belong there. **Why this step matters:** cramming everything into Tools "because that's the one I understand" is a real mistake real MCP servers make — a client that only wants to *read* your notes shouldn't have to go through a Tool call, with its own separate permission/approval model in most clients, just to read something. Getting this distinction right is the actual skill this project is teaching, more than the SQLite code in Step 2. **When you'll hit this for real:** any MCP server you build past this project — the Tool/Resource/Prompt decision comes up for every single piece of functionality you add. **Helpful background:** MCP's three building blocks — Tools, Resources, and Prompts — and how each one differs.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write a `list_all_notes(path) -> list[tuple]` function in `db.py` — a plain `SELECT` of every row, oldest or newest first, your choice. 2. In `server.py`, add `@mcp.resource("notes://all")` on a function that calls `list_all_notes()` and returns a formatted read-only string (one note per line, with its id and text) — no arguments, no side effects, nothing to "call" with parameters, just data to read. 3. Add `@mcp.prompt()` on a function `summarize_notes(start_date: str, end_date: str) -> str` that returns a prompt template string — not a summary itself, a *template* asking whichever model receives it to summarize notes created between those two dates. The server never calls a model here; it only hands back the text of the prompt. 4. Test both with the Inspector: read the `notes://all` resource and confirm it lists every note you've added so far (including ones from Step 2's restart test). Call `summarize_notes` with two dates and confirm you get back a well-formed prompt string, not an actual summary.

Your files after Step 3:

```
project_7_mcpforge_mcp_server/
├─ .env / .env.example
├─ config.py
├─ logging_setup.py
├─ exceptions.py
├─ db.py
└─ server.py
```

→ now also has `list_all_notes()`
→ adds `notes://all` Resource and `summarize_not`

Step 4 — A Real Client That Connects and Uses All Three

Project: MCPForge-Local-Notes-And-Tasks-Server — Step 4 of 4: A Real Client That Connects and Uses All Three

What this step does: builds a real MCP *client* — a standalone script that connects to your server, discovers what it offers, and actually exercises all three building blocks in one run. **What's new vs. Step 3:** a completely new file, `client.py` — nothing about `server.py` changes in this step. **What stays the same:** the server itself; this step proves it from the outside, the way a real client (Claude Desktop, your own agent) would use it, never touching its internals. **Why this step matters:** a server nobody has actually connected to and used isn't proven to work — the Inspector is a great manual testing tool, but a real client script proves the *exact* code path a future agent would use, works end to end, not just that a human clicking buttons in a UI got a sensible response. **When you'll hit this for real:** any time your own agent code (not a human) needs to use an MCP server's tools — this exact `stdio_client` / `ClientSession` shape is what a real agent's MCP integration looks like underneath. **Helpful background:** how MCP servers and clients actually connect, and why MCP matters for multi-agent systems.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write `client.py`: use `StdioServerParameters(command="python", args=["server.py"])` and `stdio_client(...)` to launch your server as a subprocess, then open a

`ClientSession` and call `await session.initialize()`. 2. Call `await session.list_tools()`, `await session.list_resources()`, and `await session.list_prompts()` — print what each returns. This is "discovered, not hardcoded" in action: your client never imports anything from `server.py` directly. 3. Call `add_note` through `session.call_tool("add_note", {"text": "..."})`, then `search_notes` the same way. Read the `notes://all` resource with `session.read_resource("notes://all")`. Get the `summarize_notes` prompt with `session.get_prompt("summarize_notes", {"start_date": "...", "end_date": "..."})`. 4. Print every result. Run the whole script start to finish and confirm all 4 calls succeed in one run — this is the full loop the Charter promised, working end to end, outside of the Inspector.

Your files after Step 4 (final):

```
project_7_mcpforge_mcp_server/
├─ .env / .env.example
├─ config.py
├─ logging_setup.py
├─ exceptions.py
├─ db.py
├─ server.py                      (unchanged from Step 3)
└─ client.py                      → standalone MCP client: list_tools/list_res
```

Final Deliverable: MCPForge-Local-Notes-And-Tasks-Server — a real MCP server exposing 2 Tools (`add_note` , `search_notes`), 1 Resource (`notes://all`), and 1 Prompt (`summarize_notes`), backed by a real SQLite database, plus a standalone client script that proves the full connect → discover → use loop works end to end.

Checklist Before You Call This Done

- [] Step 1's minimal server starts, and a client (Inspector or your own script) successfully calls `add_note`
- [] `add_note` and `search_notes` are backed by a real SQLite database file — notes survive a full server restart
- [] Empty or whitespace-only note text is rejected with a clear error, never silently stored
- [] `notes://all` is a Resource, not a third Tool — you can explain why
- [] `summarize_notes` is a Prompt, parameterized by a real date range, not a hardcoded string
- [] `client.py` connects, lists tools/resources/prompts, calls a tool, reads the resource, and uses the prompt — all in one run
- [] You can explain, for each of the 4 things this server exposes, why it's a Tool vs. a Resource vs. a Prompt, without help

Status

Not started. Track your own progress however works for you.